# Fork-per-client 系統資訊服務期中專案報告

## 1. 系統設計

本專案採用 Linux/POSIX TCP client/server 架構。伺服器預設監聽 `127.0.0.1:5050`，每次 `accept()` 成功後由父程序呼叫 `fork()`；子程序處理單一連線後結束，父程序關閉該連線描述符並繼續接受新連線。`SIGCHLD` handler 使用 `waitpid(..., WNOHANG)` 回收子程序，避免 zombie。listen backlog 設為 64；本機 10 連線的實際併發驗證由 `make test` 完成。

應用層使用一行一個命令的純文字協定：`INFO`、`PING`、`HOLD 1..30`，每個回覆以 `OK` 或 `ERR` 開頭。`INFO` 取得 hostname、作業系統、核心版本、架構和 CPU 數。`HOLD` 是可控的延遲命令，用於驗證多個子程序確實重疊執行，不是效能用途。

兩端共用的 logging、完整傳送、行接收、socket timeout、port 驗證封裝在 `libnetutil.so`。Makefile 將 client/server 動態連結此函式庫，並設定 `$ORIGIN` rpath，讓執行檔能在 `build/` 找到同目錄的 shared object。

## 2. 健壯性設計

預設模式使用 255-byte 上限與 newline framing；超長資料拒絕處理，缺少 delimiter 時 5 秒後回覆 timeout，命令需完整符合協定。client/server 皆設定 socket timeout；共用傳送函式處理部分寫入和 `EINTR`，採用 `MSG_NOSIGNAL` 避免對端關閉造成 SIGPIPE。伺服器也處理 `accept()` 中斷、fork 失敗、socket 設定錯誤與子程序回收；CLI 檢查 IPv4 位址及 1..65535 port。

`--legacy-demo` 僅用於呈現錯誤處理差異：將一次 `recv()` 當成完整訊息、不設定伺服器讀取 timeout，並允許 `PING`/`INFO` 後綴。它仍保留固定大小緩衝區，避免教學範例本身造成記憶體破壞；不可用於正式環境。

## 3. 關鍵原始碼

| 檔案 | 職責 |
|---|---|
| `src/server.c` | listener、`accept()`/`fork()`、子程序生命週期、請求驗證、系統資訊收集 |
| `src/client.c` | 命令列解析、TCP 連線、請求傳送及逾時回覆處理 |
| `src/netutil.c` | 動態共用的 log、`send_all()`、line receive、socket timeout 和 port parser |
| `include/netutil.h` | client/server 共用函式宣告與封包大小常數 |
| `Makefile` | `.so`、client、server 建置及測試/抓包入口 |
| `scripts/test.sh` | 三組錯誤輸入對照及 100 client runtime proof |
| `scripts/capture.sh` | tcpdump loopback 擷取與封包文字解碼 |

Debug 有兩層：編譯期 `DEBUG=1/0` 決定是否把 DEBUG 日誌編入；執行期 `--verbose` 控制已編入的 DEBUG 日誌是否輸出。WARN/ERROR/INFO 不會被關閉。

## 4. 建置與執行

```
make clean && make DEBUG=1
./build/server --verbose 127.0.0.1 5050
./build/client --verbose 127.0.0.1 5050 INFO
./build/client 127.0.0.1 5050 PING
make test
```

`make test` 產生 `artifacts/robustness-examples.txt`、`artifacts/concurrency-proof.txt`、server log 及 100 個 client 回覆檔。以 `make clean && make DEBUG=0` 可驗證編譯期關閉 DEBUG。

## 5. 健壯性實驗（至少三例）

執行 `make test` 會在兩個本機 port 啟動 robust server 與 `--legacy-demo` server，逐項送出同一個原始 TCP payload。實際輸出寫入 `artifacts/robustness-examples.txt`；下表描述預期觀察，提交前請以產生的 log 核對並附上終端截圖。

| 案例 | 輸入 | Robust 模式 | Legacy demo |
|---|---|---|---|
| 1. 未送 newline | `PING`（連線仍開啟） | 5 秒後回覆 `ERR request timed out` | 把單次 recv 當完整命令，錯誤地回覆 `OK PONG` |
| 2. 超長封包 | `PING` + 300 個 `x` + newline | 回覆 `ERR request too long` | 只讀固定緩衝區前段，錯誤地回覆 `OK PONG` |
| 3. 命令尾隨垃圾 | `INFO garbage` | 回覆 `ERR invalid request` | 前綴比對通過，錯誤地回傳 system info |

以上結果顯示 framing、長度上限、timeout 和完整命令驗證如何避免不完整或無效輸入被當成有效命令。Legacy 測試僅限本機 loopback。

## 6. 10 客戶端併發證明

`make test` 預設同時啟動 100 個 client，各自要求伺服器執行 `HOLD 3`。本次實測 100/100 成功，牆鐘時間為 3.24 秒；若逐一處理則約需 300 秒。伺服器整場測試共記錄 109 筆 fork（包含基本命令與異常輸入測試）；併發區段列出 100 個不同的 worker PID。完整輸出見 `artifacts/concurrency-proof.txt`，每個 client 的回覆另存於 `artifacts/concurrent-client-*.txt`。可用環境變數 `CLIENT_COUNT` 調整測試數量。

## 7. 封包擷取與分析

本次使用 tcpdump 在 loopback `lo` 擷取 TCP port 25060，產生 `artifacts/client-server.pcap`。tcpdump 統計 20 個封包寫入 pcap、40 個封包符合 filter、0 個封包遭 kernel 丟棄。擷取內容包含兩條 client/server 連線：來源埠 52026 傳送 5-byte `PING`，伺服器回覆 8-byte `OK PONG`；來源埠 52040 傳送 5-byte `INFO`，伺服器回覆 86-byte system information。兩條連線皆可辨認 SYN、SYN-ACK、ACK 三向交握、資料與 ACK，以及 FIN 關閉流程。`tcpdump -nn -A -r artifacts/client-server.pcap 'tcp port 25060'` 已確認 payload 可讀；協定未加密是本教學範例的限制。

`artifacts/packet-analysis.txt` 保存封包摘要。提交時附上該摘要的終端截圖，以及 Wireshark 開啟 `client-server.pcap`、以 `tcp.port == 25060` 過濾並選取含 `INFO` 回覆封包的畫面。埠號可依 `CAPTURE_PORT` 調整。

# 8. 截圖與影片提交清單

- 截圖 A：編譯成功、`INFO` 回覆與 server `--verbose` log。
- 截圖 B：`artifacts/concurrency-proof.txt` 的 100/100、總時間及 fork worker PID。
- 截圖 C：三組 robust/legacy 對照輸出。
- 截圖 D：Wireshark 的 TCP handshake 與明文 INFO/OK 封包，或 tcpdump 解碼畫面。
- 影片：依 `report/VIDEO_SCRIPT.md` 錄製約 2 分鐘，展示 build、INFO、併發與一個健壯性差異。

截圖與 pcap 應由學生在自己的課堂電腦執行後保存；不可用預期輸出冒充實測截圖。